TECHNICAL NOTE 10

AGENTIC FOUNDATIONS & MULTI-AGENT ARCHITECTURES

LLMs, ReAct, RAG, Tool Use, AutoGen, CrewAI & Vector Stores

Corporate Intelligence Course
Universidad Francisco de Vitoria
Prof. Cesar Moreno Pascual PhD.

2025-2026 Academic Year

1. From Static Analysis to Agentic Intelligence

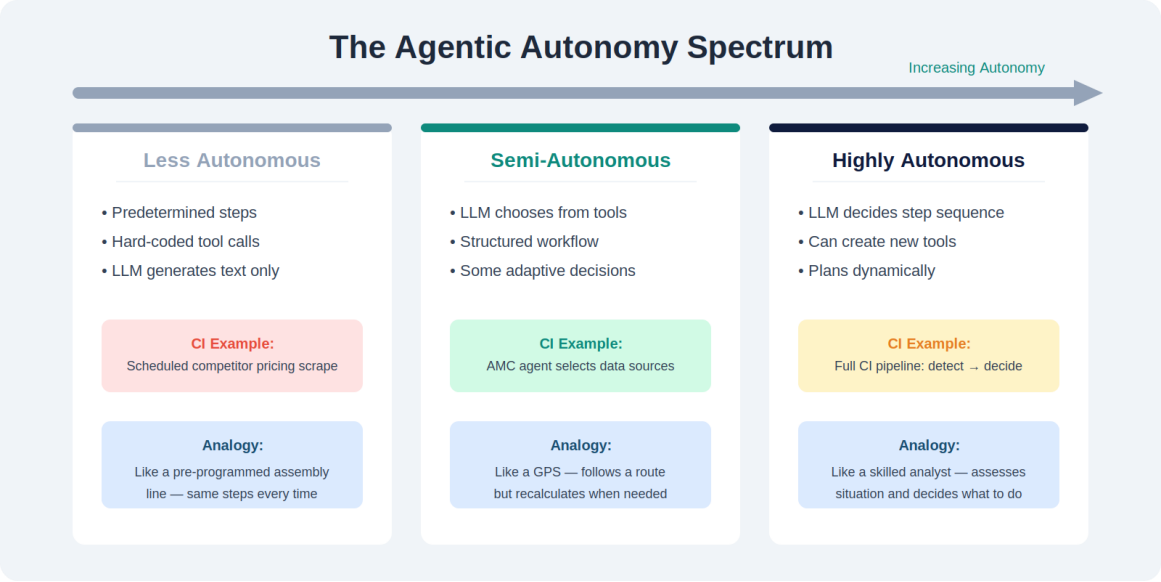
The CI frameworks you have learned in this course (AMC, Exchange Value, ECOMO, War Gaming, Blind Spots) provide powerful analytical structures. However, applying them manually is time-consuming, data-intensive, and prone to the very cognitive biases TN03 warns about. Agentic AI systems offer a paradigm shift: they can **autonomously execute the multi-step analytical workflows** your frameworks demand, while continuously ingesting real-time data from OSINT sources.

> **KEY INSIGHT: The Agentic Advantage for CI**
>
> Andrew Ng popularised the term 'agentic' to describe AI workflows that iterate, reflect, and use tools rather than generating a single response. On a coding benchmark (HumanEval), wrapping GPT-3.5 in an agentic workflow outperformed even GPT-4 used in a non-agentic manner. The lesson: iterative AI workflows dramatically outperform one-shot prompting for complex analytical tasks like competitive intelligence.

1.1 What Makes a System 'Agentic'?

An agentic workflow is a process where an LLM-based application executes multiple steps to complete a task, rather than generating a single output from a single prompt. The degree of autonomy exists on a spectrum:

Autonomy Level	Characteristics	CI Application Example
Less Autonomous	Predetermined steps, hard-coded tool calls, LLM generates text only	Scheduled competitor pricing scrape with a fixed report template
Semi-Autonomous	LLM chooses from predefined tools, follows a structured workflow	AMC analysis agent that selects which data sources to query
Highly Autonomous	LLM decides step sequence, can create new tools, plans dynamically	Full CI pipeline that identifies phenomena, collects data, and generates strategic recommendations



Easy Example

Think of three ways to get dinner:

(1) Less Autonomous: A microwave meal — you follow the exact instructions on the box, every time the same.

(2) Semi-Autonomous: A recipe kit (like HelloFresh) — you have ingredients and a guide, but you make small decisions about seasoning and timing.

(3) Highly Autonomous: A professional chef — they assess what's available, plan the menu, adapt if an ingredient is missing, and create something new.

CI Connection
Your course frameworks (AMC, Exchange Value, ECOMO, War Gaming, Blind Spots) provide the analytical structure. Agentic AI provides the computational engine to execute them autonomously at scale — from Level 1 (scripted reports) to Level 3 (full CI pipeline from phenomenon detection to strategic recommendation).

1.2 Four Design Patterns for CI Agents

Andrew Ng identified four fundamental design patterns that underpin all agentic AI systems. Each maps directly to a CI framework you have already learned in this course

Figure 2.1 — Four Design Patterns with CI framework mappings and everyday analogies

Four fundamental design patterns underpin all agentic CI systems:

Design Pattern	Mechanism	CI Framework Connection
Reflection	Agent critiques its own output, identifies gaps, and iterates	Maps to Devil's Advocacy and Pre-Mortem (TN03) as automated bias checks
Tool Use	Agent calls external APIs, databases, and web scrapers	Maps to OSINT data collection (USAspending, EDGAR, PatentsView)

Design Pattern	Mechanism	CI Framework Connection
Planning	The agent decomposes complex tasks into ordered sub-steps	Maps to the CI analytical workflow: Phenomenon > AMC > Scenarios > ECOMO
Multi-Agent	Specialised agents collaborate on complex tasks	Maps to War Gaming roles: Red Team, Blue Team, Control, Reporter

Easy Examples for Each Pattern

Reflection — Like re-reading your own essay

You write a first draft, then re-read it critically: 'This paragraph has no evidence.' You revise and improve. The agent does the same with its AMC analysis: generates scores, then checks whether each score is cited.

Tool Use — Like using a calculator during a maths exam

You could try to do complex calculations in your head, but it's faster and more accurate to use a calculator. The agent uses external tools (EDGAR, PatentsView, USAspending) instead of relying on its training data.

Planning — Like planning a holiday

You don't just show up at the airport. You research destinations, book flights, reserve hotels, and plan activities. The agent decomposes 'Should we enter the EV battery market?' into: identify competitors → collect data → score AMC → generate scenarios → calculate Exchange Value → ECOMO decision.

Multi-Agent — Like a hospital surgery team

A surgeon doesn't also do anaesthesia and nursing. Each specialist has a role. Similarly, a CI Collector agent gathers data, an AMC Analyst scores competitors, a Verifier checks sources, and a Reporter writes the brief. The Orchestrator coordinates them like the lead surgeon

2. LLMs as Competitive Intelligence Reasoners

Large Language Models serve as the cognitive engine of agentic CI systems. Understanding their capabilities and limitations is essential for designing effective workflows.

2.1 CI-Relevant Capabilities

Natural Language Understanding: Process unstructured text from SEC filings, news articles, earnings call transcripts, patent applications, and social media.

Multi-Document Synthesis enables the integration of information from USAspending.gov, EDGAR, and PatentsView to produce coherent competitor profiles.

Pattern Recognition identifies trends and anomalies across large datasets while being automated.

Report Generation creates structured intelligence briefs from raw OSINT data.

2.2 Critical Limitations for CI

These limitations directly inform the architectural choices covered in the rest of this note:

Limitation	CI Risk	Mitigation (covered in this note)
Knowledge Cutoff	Missing recent competitor actions, regulatory changes	RAG + Real-time tool use (Section 4)
Hallucination	Fabricated revenue figures, non-existent patents, and wrong executive names	Anti-hallucination patterns (TN11 Section 1)
Context Length	Cannot process entire 10-K filings in one pass	Document chunking in RAG (Section 4)
Reasoning Depth	A complex multi-step AMC analysis may fail	ReAct framework with iterative loops (Section 3)
Source Attribution	Claims without verifiable sources undermine CI credibility	Mandatory citation architecture (TN11 Section 4)

3. The ReAct Framework for CI

Yao et al. (2023) introduced ReAct (Reasoning and Acting), a framework that synergises reasoning traces with tool use. ReAct interleaves three types of steps: **Thought** (verbal reasoning about what to do next), **Action** (tool call such as web search, API request, or database query), and **Observation** (result from the action, fed back into reasoning).

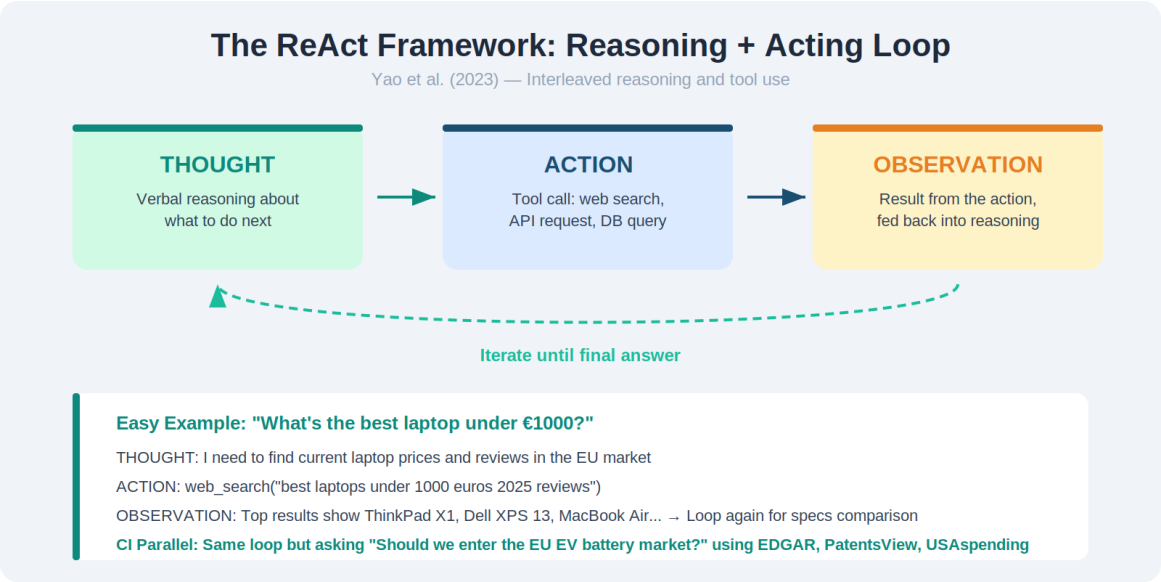


Figure 3.1 — The ReAct loop: Thought → Action → Observation, iterating until the final answer

Easy Example

Imagine you're trying to decide what laptop to buy:

THOUGHT: 'I need to find laptops under €1000 with good reviews.'

ACTION: You Google 'best laptops under 1000 euros 2025.'

OBSERVATION: Top results show ThinkPad X1, Dell XPS 13, MacBook Air.

THOUGHT: 'Now I need to compare battery life and weight.'

ACTION: You search for detailed specs of each model.

OBSERVATION: ThinkPad has a 15h battery (1.2kg); Dell has a 12h battery (1.3kg)...

You keep looping until you have enough information to decide.

3.1 ReAct Applied to the CI Analytical Workflow

The ReAct loop maps naturally to the integrated CI framework from TN01. Consider how an agent would process the question: 'Should we enter the European EV battery market?'

Step	ReAct Phase	CI Framework Stage	Agent Action
1	Thought	Phenomenon Identification	I need to identify who the key competitors are in EU EV batteries
2	Action	OSINT Collection	call_tool(web_search, 'EU EV battery manufacturers market share 2025')
3	Observation	Data Ingestion	Top players: CATL, Samsung SDI, Northvolt, ACC...
4	Thought	AMC Analysis	Now I need to assess each competitor's Awareness, Motivation, and Capability.
5	Action	AMC Data Gathering	call_tool(sec_edgar, '10-K filings CATL Samsung SDI')
6	Observation	Evidence Integration	CATL: Revenue EUR 45B, R&D spend EUR 3.2B...
7	Thought	Scenario Generation	Based on AMC scores, I can generate 3 scenarios for market entry
8	Action	Exchange Value Calc	call_tool(calculator, exchange_value_model)
9	Final Answer	Strategic Recommendation	Synthesise Go/No-Go recommendation with ECOMO analysis

CI FRAMEWORK CONNECTION

The ReAct loop is the computational implementation of the Strategic Analysis Workflow from TN01 (Figure 5.1). Each Thought-Action-Observation cycle corresponds to a step in the pathway: Contemplated Action > AMC Analysis > Repertoire Analysis > Scenario Generation > Exchange Value > Expected Value > Strategic Decision.

3.2 Advantages for CI

Transparency: Reasoning traces provide an audit trail for intelligence claims, addressing the SCIP Code of Ethics requirement for traceable analysis. **Grounding:** Tool use grounds responses in current, verifiable OSINT data rather than training data. **Flexibility:** The agent can adapt its analytical strategy based on intermediate results, just as a human analyst would pivot when initial data reveals unexpected patterns. **Error Recovery:** Failed API calls or contradictory data trigger re-evaluation rather than silent failure.

4. Retrieval-Augmented Generation (RAG) for CI

RAG (Lewis et al., 2020) grounds LLM responses in current, authoritative data by retrieving relevant context before generation. For CI, RAG is essential because competitive landscapes change daily, and LLM training data has a cutoff.

Easy Example

Think of it like an open-book exam versus a closed-book exam:

Without RAG (closed-book): The student answers from memory. They might remember wrong facts, confuse dates, or hallucinate details. This is dangerous for CI, where accuracy matters.

With RAG (open-book): The student can look up their notes before answering. They find the exact revenue figure, the correct patent filing date, and the precise contract amount. Their answer is grounded, accurate, and citable.

> **CI Application**
> The agent stores competitor filings, patents, press releases, and market reports in ChromaDB collections. When asked about Palantir's government revenue trend, it retrieves the actual 10-K data before generating its AMC analysis — no hallucination, full citations.

4.1 RAG Architecture

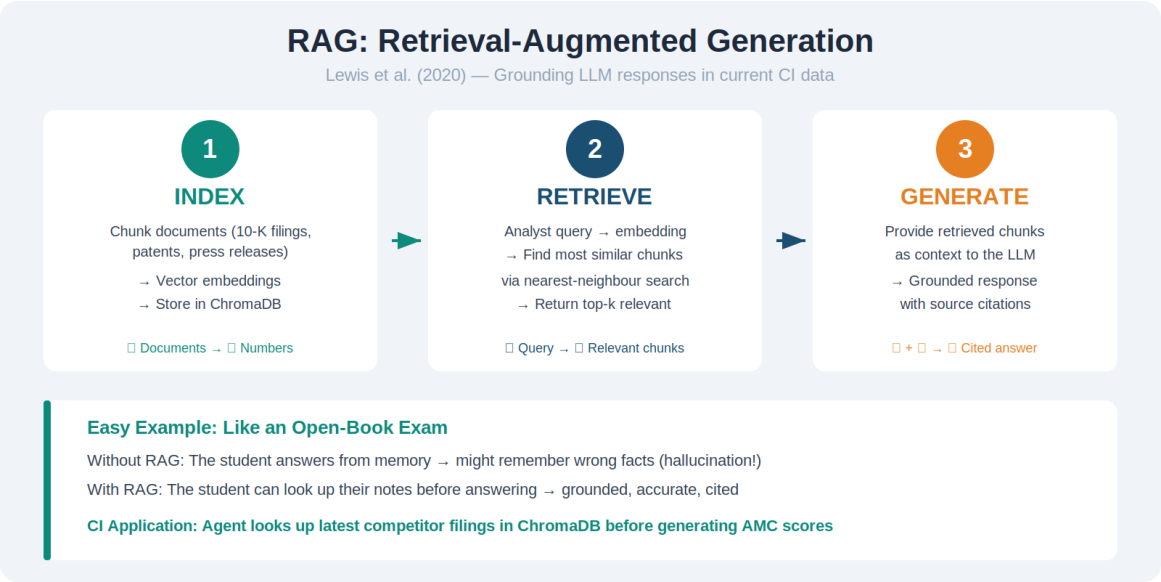


Figure 4.1 — The three-step RAG process: Index → Retrieve → Generate with ChromaDB

Step 1 - Indexing: Chunk documents (competitor filings, press releases, patent applications) into segments of 256-512 tokens. Create vector embeddings for each chunk. Store in a vector database such as ChromaDB, Pinecone, or Weaviate.

Step 2 - Retrieval: Convert the analyst's query to an embedding. Find the most similar chunks via approximate nearest-neighbour search. Return the top-k relevant chunks.

Step 3 - Generation: Provide retrieved chunks as context to the LLM. The model generates a response grounded in current documents, with citations tracking which documents contributed to each claim.

4.2 CI Memory Architecture with ChromaDB

ChromaDB provides persistent vector storage that serves as the agent's institutional memory. For CI applications, organise collections by analytical function:

Collection	Contents	Update Frequency	CI Function
competitor_intel	Per-competitor profiles, filings, news	Daily	AMC scoring, Repertoire tracking
market_trends	Industry reports, market data, macro indicators	Weekly	Scenario planning driving forces
internal_strategy	Company strategy docs, previous decisions	Monthly	Reference scenario for Exchange Value
historical_archive	Past intelligence reports, prediction outcomes	Continuous	Learning cycle feedback (TN01 Fig 5.1)

4.3 Deep Dive: What Is an Embedding and Why ChromaDB?

The RAG process above depends on a fundamental technology that is often found confusing: embeddings. This section explains exactly what an embedding is, why it matters, and why we need a specialised database like ChromaDB to store them.

The Problem: Computers Cannot Compare Meaning

A computer stores text as characters: “Palantir DoD contract” is just a sequence of letters to a machine. It has no way of knowing that this phrase is closely related to “defence AI government spending” — they share zero words in common. Traditional databases can only match exact keywords. If you search for “defence AI spending”, a keyword search would never return a document titled “Palantir DoD contract” — even though they are about the same topic.

The Solution: Embeddings Turn Meaning into Numbers

An embedding is a list of approximately 1,500 numbers (called a “vector”) that captures the meaning of a piece of text. An embedding model has been trained on billions of sentences, so it learns that semantically similar texts produce similar numbers. After conversion: “Palantir DoD contract” becomes [0.82, 0.91, 0.03, ...] and “defence AI government spending” becomes [0.79, 0.88, 0.05, ...]. These two vectors are very close in mathematical space because their meanings are related, even though they use completely different words. Meanwhile, “pasta recipe” becomes [0.12, 0.08, 0.94, ...], which is far away because its meaning is completely unrelated.

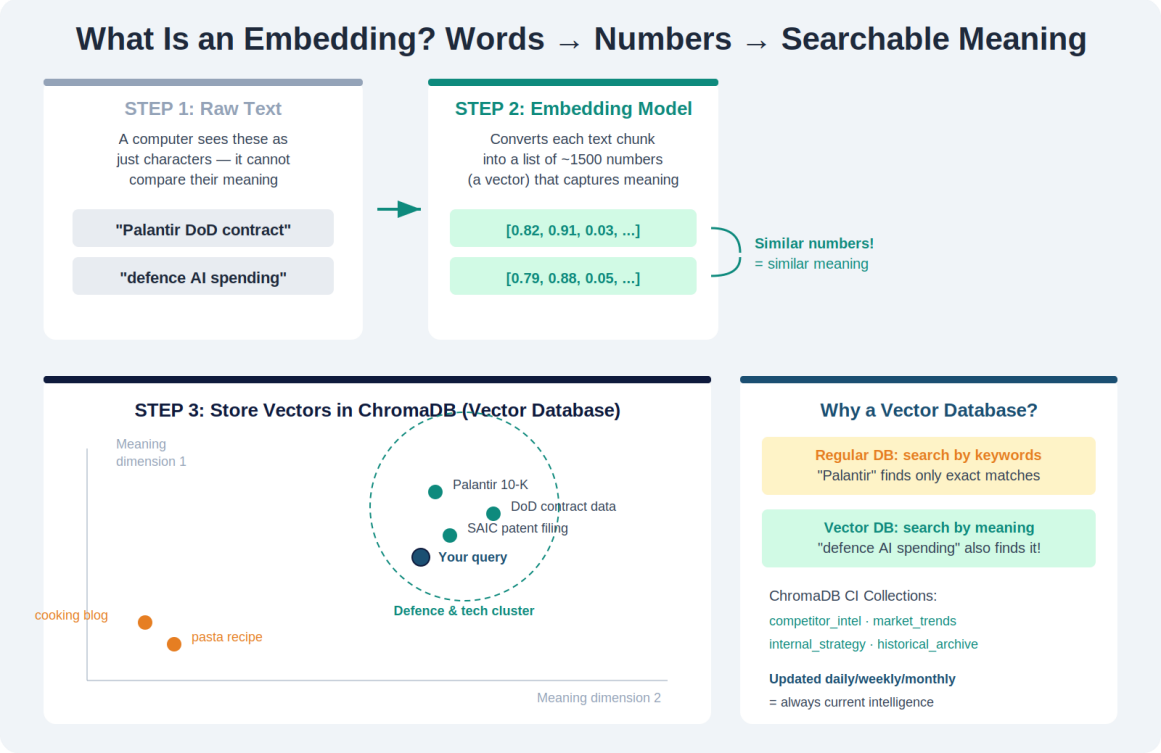


Figure 4.2 — From raw text to searchable vectors: how embeddings enable meaning-based retrieval in ChromaDB

Easy Example: The Meaning-Based Library

Imagine a library where books are not shelved alphabetically or by author, but by topic similarity. A book about "military AI contracts" would sit right next to "Palantir defence revenue" — even though those titles share zero words — because they are about the same topic. When you walk in asking about "DoD technology spending", the librarian points you straight to that shelf because your question means the same thing. That librarian is the embedding model, and those shelves are ChromaDB.

Why ChromaDB Instead of a Normal Database?

A regular database (SQL, Excel) stores data in rows and columns and searches by exact keyword match. If you search for "defence AI spending", it only finds documents containing those exact words. A vector database like ChromaDB stores the number vectors produced by the embedding model and searches by mathematical distance — it finds documents whose meaning is closest to your query, regardless of which specific words they use. This is why the CI agent can ask "What is Palantir's government revenue trend?" and ChromaDB returns the actual 10-K filing data, even if the filing never contains the phrase "government revenue trend" anywhere in its text.

The Technical Process in Three Sentences

(1) When we index a document, the embedding model converts each chunk of text into a vector of ~1,500 numbers and stores it in ChromaDB alongside the original text and metadata (source URL, date, company name).

(2) When the CI agent asks a question, that question is also converted to a vector using the same embedding model.

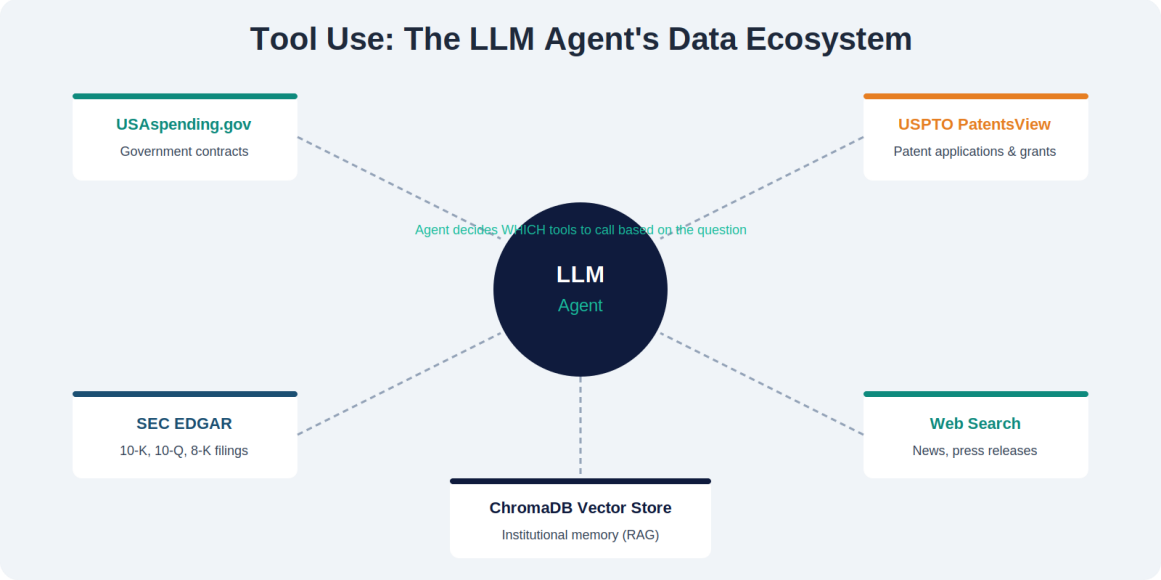
(3) ChromaDB performs an approximate nearest-neighbour search — it finds the stored vectors that are mathematically closest to the query vector — and returns the corresponding original text chunks. Those chunks serve as the context the LLM uses to generate a grounded, cited answer.

Why This Matters for CI

> Without embeddings and a vector database, the CI agent would have to rely on its training data (which has a cutoff date) or on exact keyword searches (which miss semantically related documents). With RAG powered by ChromaDB, the agent always retrieves the most current, relevant intelligence before answering — grounding every claim in verifiable sources. This is what eliminates hallucination and makes the difference between a toy demo and a production CI system.

5. Tool Use and APIs for CI

Modern LLMs can use external tools through function calling, enabling integration with the OSINT data sources you have used throughout this course. The key insight is that the agent decides which tools to call based on the analytical question — this is what makes it 'agentic' rather than scripted.



Easy Example

Think of a journalist writing an investigative article. They don't just sit at their desk and write from memory. They call sources, look up public records, check financial databases, and verify facts. The LLM agent does the same thing, but computationally: it calls USAspending for contracts, EDGAR for financials, PatentsView for R&D intelligence, and web search for recent news.

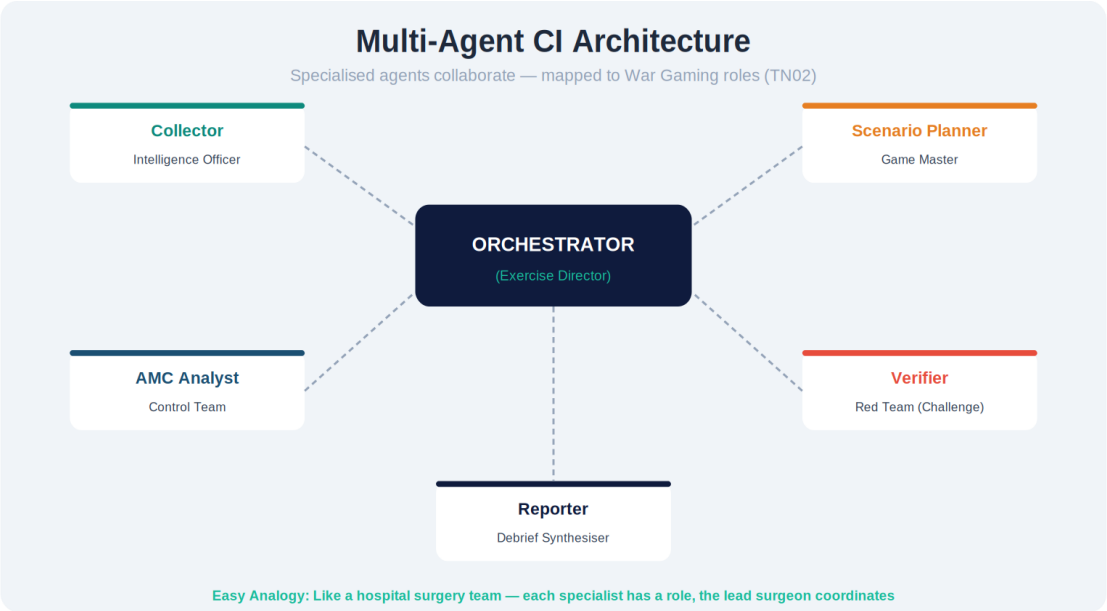
> **Key Insight**
> Each tool maps to a specific stage in the CI analytical workflow. The agent is not randomly calling APIs — it calls USAspending when assessing Capability (AMC), PatentsView when mapping Awareness of technology trends, and web search when identifying Phenomena. This mapping is what connects agentic AI to the frameworks you have learned.

5.1 CI Tool Categories

Tool Category	Examples	CI Framework Stage
Web Search	Google Search API, Tavily, DuckDuckGo	Phenomenon identification, news monitoring
Government APIs	USAspending.gov, SAM.gov, SEC EDGAR	Capability assessment (AMC), financial analysis
Patent APIs	USPTO PatentsView, Google Patents	R&D intelligence, technology landscape mapping
Web Scraping	Playwright, BeautifulSoup	Competitor pricing, product changes, messaging shifts
Document Processing	PDF parsing, Excel reading	10-K analysis, contract parsing
Calculation	Python code execution, financial models	Exchange Value, ECOMO NPV calculations

6. Multi-Agent Architectures for CI

Complex CI tasks often exceed the capabilities of single agents. Multi-agent systems enable specialisation, parallelisation, error isolation, and scalability — mirroring how CI teams and War Gaming exercises operate in practice.



Easy Example

Think of a Formula 1 pit stop: the wheel changers, jack operators, fuel handlers, and wing adjusters each have a highly specialised role. They work in parallel, coordinated by the crew chief. If one person makes a mistake, it doesn't crash the whole team — others keep working.

In the CI multi-agent system, the Collector gathers data, the AMC Analyst scores competitors, the Scenario Planner generates scenarios, the Verifier checks for hallucinations, and the Reporter writes the brief. The Orchestrator coordinates everything, just like the crew chief.

> **War Gaming Connection**
> This architecture is the computational implementation of War Gaming from TN02. The Blue Team becomes a Strategy Agent; Red Teams become competitor agents (each with that competitor's AMC profile as its system prompt); Control becomes the Orchestrator; and the Debrief Facilitator becomes the Reporter. A multi-round war game that takes days with humans can run in minutes with agents.

6.1 Why Multi-Agent for CI?

Specialisation: Each agent focuses on specific tasks (collection, analysis, verification), just as a CI department has distinct roles. **Parallelisation:** Multiple agents can query USAspending, EDGAR, and PatentsView simultaneously. **Error Isolation:** A failed patent API call does not cause the financial analysis agent to crash. **Scalability:** Add new agents as the analytical scope expands to new verticals or data sources.

6.2 CI Agent Roles Mapped to Frameworks

Agent Role	Function	CI Framework Mapping	War Game Equivalent
Collector	Gathers raw OSINT data from APIs and web	Data inputs (TN01 Fig 5.1)	Intelligence Officer

Agent Role	Function	CI Framework Mapping	War Game Equivalent
AMC Analyst	Scores Awareness, Motivation, Capability	AMC Framework (TN01 Sec 2)	Control Team assessment
Scenario Planner	Generates and evaluates competitive scenarios	Shell Method + Exchange Value	Game Master / Facilitator
Verifier	Cross-references sources, assigns confidence	Anti-hallucination (TN11)	Red Team challenge function
Reporter	Generates executive briefs with citations	Decision outputs (TN01 Fig 5.1)	Debrief synthesiser
Orchestrator	Coordinates workflow, routes tasks	CI analytical workflow	Exercise Director

6.3 AutoGen: Multi-Agent Conversations

Wu et al. (2023) introduced AutoGen, which enables multi-agent conversations in which specialised agents collaborate. Core concepts include **Conversable Agents** (agents that send and receive messages), **Conversation Patterns** (two-agent, group chat, hierarchical), and **Human-in-the-Loop** (human participates as an agent for high-stakes decisions).

Example workflow for a competitive analysis using AutoGen:

Step	Actor	Action
1	User	Request an analysis of the competitor's new product launch
2	Orchestrator	Assigns the Collector to gather press releases and product pages
3	Collector	Returns raw data from web scraping and API calls
4	Orchestrator	Routes data to AMC Analyst
5	AMC Analyst	Produces AMC scores and response predictions
6	Orchestrator	Sends to Scenario Planner for Exchange Value analysis
7	Scenario Planner	Generates 3 scenarios with probabilities and economic outcomes
8	Verifier	Cross-references claims, flags LOW confidence findings
9	Reporter	Generates an executive brief with a Go/No-Go recommendation

6.4 CrewAI: Role-Based Orchestration

CrewAI provides a higher-level abstraction for role-based agent orchestration, built around three core components:

Component	Description	CI Application
Agent	Role + Goal + Backstory + Tools	CI Analyst with access to EDGAR, PatentsView tools
Task	Description + Expected Output + Dependencies	Complete AMC analysis for 5 competitors

Component	Description	CI Application
Crew	List of Agents + Tasks + Process (sequential/hierarchical)	Full CI pipeline from data collection to recommendation

8. EU AI Act Compliance for CI Systems

CI agents using AI must consider the EU AI Act requirements, particularly for high-risk applications that affect strategic business decisions.

Risk Level	CI Application	Requirements
Minimal Risk	Market monitoring, automated report generation	No specific obligations, but documentation is recommended
Limited Risk	Customer-facing CI chatbots, competitor dashboards	Transparency: users must know they are interacting with AI
High Risk	CI systems influencing employment decisions, major investments	Full documentation, testing, human oversight, conformity assessment

9. References

Yao, S. et al. (2023). ReAct: Synergising Reasoning and Acting in Language Models. ICLR 2023.

Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

Wu, Q. et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. Microsoft Research.

CrewAI Documentation: https://docs.crewai.com/

ChromaDB Documentation: https://docs.trychroma.com/

LangChain Documentation: https://python.langchain.com/

Ng, A. (2025). AI Agentic Workflows. DeepLearning.AI.

Chen, M.J. & Miller, D. (2012). Competitive Dynamics: Themes, Trends, and a Prospective Research Platform. Academy of Management Annals.